

PENETRATION TESTING TRAINING EXERCISE

WEEK 1 — ATTACK SURFACE MAPPING

OWASP Juice Shop — Reconnaissance & Attack Surface Assessment



A hands-on look at how an attacker would map this application — every page, API and hidden path — without exploiting a thing.

25+ Routes & pages discovered	20+ API endpoints catalogued	6 HTTP methods confirmed	1 + 3 Critical + High findings
---	--	------------------------------------	--

TARGET APPLICATION	OWASP Juice Shop (bkimminich/juice-shop)
TEST ENVIRONMENT	Kali Linux 2026.3 (VM) — Docker — http://localhost:3000
ASSESSMENT TYPE	Black-box Reconnaissance / Attack Surface Discovery (No Exploitation)
REPORT DATE	October 2026
CLASSIFICATION	Confidential — Training Use Only
PREPARED BY	Muhammad Ali Mobeen — Cybersecurity Intern
ORGANIZATION	Cronova Solutions — 8-Week Cybersecurity Internship Program
REFERENCE NO.	CSL-INT-010
CONTACT	cronovasolutions.tech contact@cronovasolutions.tech

LIABILITY & SCOPE DISCLAIMER

This report covers reconnaissance and attack-surface discovery that I carried out only against a self-hosted, local copy of OWASP Juice Shop. OWASP publishes Juice Shop as a deliberately vulnerable application for training and learning. I did not access, test or affect any production system, third-party infrastructure or real user data at any point.

Everything described here was done inside an isolated Kali Linux virtual machine that I fully controlled, as part of an authorized training assignment. I did not exploit anything, extract data, run denial-of-service tests or do anything destructive. I stayed within what the exercise brief allowed: reconnaissance and non-destructive verification only.

Neither I nor anyone who distributes this report accepts liability for what third parties do with the information, techniques or findings in it against systems they have no explicit written permission to test. Scanning, enumerating or testing systems you don't own or aren't authorized to test is illegal in most countries (for example under the US Computer Fraud and Abuse Act, the UK Computer Misuse Act and similar local laws).

This document is for educational purposes only and was written as part of a structured cybersecurity training program. The findings reflect the known, intentional weaknesses of the OWASP Juice Shop training platform. They say nothing about the security of any other system.

Executive Summary

This report covers a structured, black-box attack-surface mapping exercise against a self-hosted copy of OWASP Juice Shop, a deliberately vulnerable web application that OWASP maintains for security training. My goal was not to exploit anything. It was to find, catalogue and prioritize everything an outsider could observe about the application (pages, routes, APIs, technologies and client-side files), the same way a real attacker would during the reconnaissance phase of an engagement.

I combined automated tools (Nmap, Gobuster and WhatWeb) with manual browsing, browser developer-tools inspection, and direct HTTP/cURL requests to the application's REST and Swagger-documented APIs. That gave me the full technology stack: an Angular front end, a Node.js/Express back end, Sequelize ORM over SQLite, and JWT-based authentication. I found more than 20 distinct API endpoints covering authentication, users, products, baskets and challenges. I also found several hidden or non-obvious paths, including a legacy FTP directory, exposed encryption-key files and a hidden Score Board. On top of that, there were multiple access-control weaknesses: a publicly reachable administration panel, an Insecure Direct Object Reference (IDOR) on basket data, and authorization that was enforced inconsistently across sensitive endpoints.

Overall, the attack surface is broad but shallow. Almost every meaningful function can be reached with no login or with a low-privilege session, and the client-side bundle gives away more about the backend than a typical production application would. The rest of the report follows the order I worked in: discoveries by category, a single attack-surface inventory, an architecture-level diagram, and finally a recommended order of investigation for a later, authorized penetration test.

Key Numbers at a Glance

25+ distinct routes and pages discovered (public, authentication-related, administrative and hidden)

20+ API endpoints catalogued across 5 functional groups (Auth, User, Product/Basket, Challenges and Admin)

6 HTTP methods confirmed available (GET, HEAD, POST, PUT, PATCH and DELETE) through CORS pre-flight checks

1 Critical and 3 High-severity access-control findings during discovery (IDOR, admin panel exposure, unauthenticated data endpoints and exposed secrets)

Part 1 — Environment Setup

I deployed OWASP Juice Shop locally inside a Kali Linux 2026.3 virtual machine, using the official Docker image. This met the "Local Virtual Machine" deployment option and kept the target completely isolated from any external network, so there was no risk to third-party systems. It was still a real, fully working instance to run reconnaissance against.

1.1 Deployment Details

Parameter	Value
Host OS / Environment	Kali Linux 2026.3 (Rolling), VirtualBox VM
Deployment Method	Docker (bkimminich/juice-shop official image, latest tag)
Run Command	sudo docker run -d -p 3000:3000 bkimminich/juice-shop
Application URL	http://localhost:3000
Image Size	~334 MB
Verification	docker ps showed the container Up with port 3000 bound; the application was reachable and fully usable in Firefox

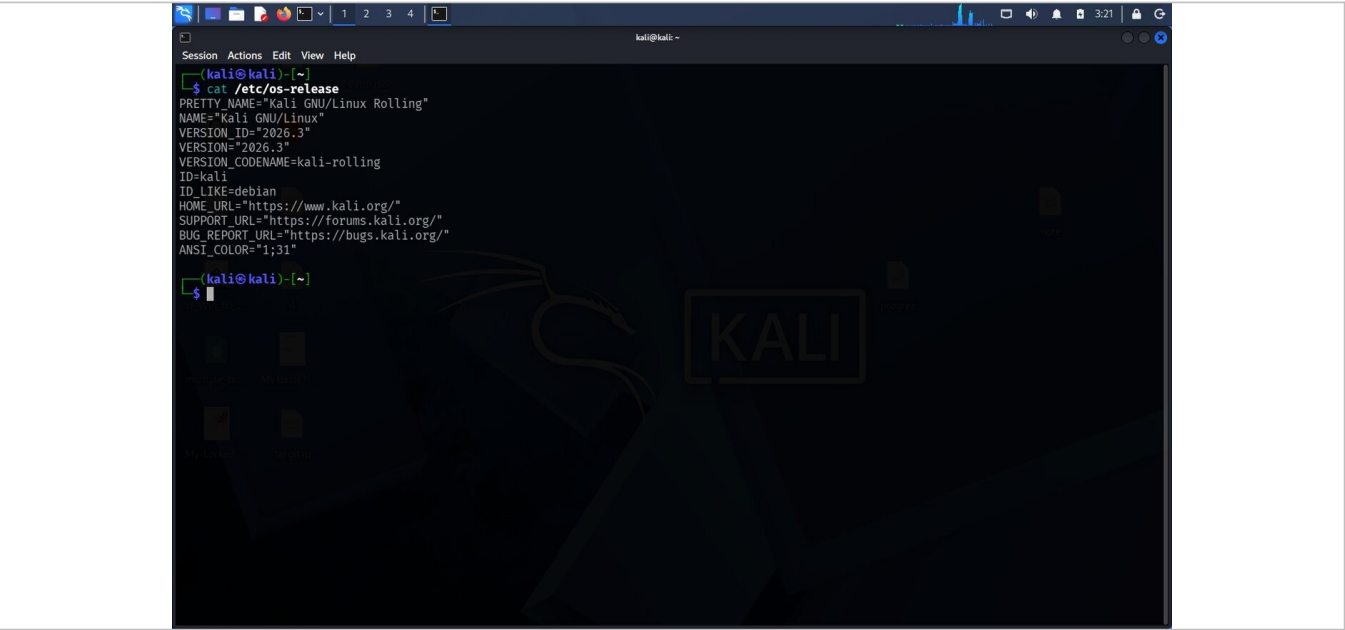


Figure 1.1 — Host OS fingerprint (/etc/os-release), confirming the Kali Linux 2026.3 test environment.

```

kali@kali: ~
Session Actions Edit View Help

(kali@kali)-[~]
$ sudo docker run -d \
  --name juice-shop \
  --restart unless-stopped \
  -p 3000:3000 \
  bkimminich/juice-shop
d8adb5a9997555d31ce9481a9d35df3c1a42e9c1317d47f09a4e214066092bf6

(kali@kali)-[~]
$ sudo docker ps
CONTAINER ID   IMAGE      NAMES      COMMAND                  CREATED        STATUS        PORTS
d8adb5a99975   bkimminich/juice-shop  "/nodejs/bin/node /j_..." About a minute ago Up 59 seconds  0.0.0.0:3000->3000/tcp,
[::]:3000->3000/tcp   juice-shop

```

Figure 1.2 — docker ps output showing the Juice Shop container running with port 3000 published.

```

kali@kali: ~
Session Actions Edit View Help

tid":2,"id":4,"quantity":2,"createdAt":"2026-09-27T08:25:30.337Z","updatedAt":"2026-09-27T08:25:30.337Z"}}}}

(kali@kali)-[~]
$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:3d:4d:96 brd ff:ff:ff:ff:ff:ff
   inet 10.0.0.4/24 brd 10.0.0.255 scope global dynamic noprefixroute eth0
       valid_lft 347sec preferred_lft 347sec
   inet6 fe80::5300:afaf:3349:1758/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: docker0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
   link/ether ee:94:ab:d2:d9:cd brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
   inet6 fe80::ec94:abff:fed2:d9cd/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
4: veth60848fe9if2: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue master docker0 state UP group defau
   lt
   link/ether da:16:63:fd:cd:81 brd ff:ff:ff:ff:ff:ff link-netnsid 0
   inet6 fe80::d816:63ff:fedd:cd81/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever

(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo

```

Figure 1.3 — Network interface configuration of the test VM, confirming an isolated local test network.

One small hiccup is worth mentioning for transparency. My first docker run attempt failed because an earlier container was still holding port 3000 ("Bind for 0.0.0.0:3000 failed: port is already allocated"). I stopped the old container and started the image again on a clean binding. This was a routine environment issue and not an application vulnerability, but I kept it in the evidence so the setup record is complete and honest.

Part 2 — Reconnaissance & Attack-Surface Discovery

2.1 Web Application Discovery

I used manual navigation, directory brute-forcing (Gobuster) and port scanning (Nmap) together to map every reachable page and route, without leaning on any predefined list of endpoints.

Publicly Accessible Pages

- Home / product catalogue — / and /#/
- Search — client-side query routed through /#/search
- Product detail pages — opened from the catalogue grid
- Contact / Customer Feedback — /#/contact
- About Us, Privacy Policy, Terms of Use (footer-linked static content)

Authentication-Related Pages

- Login — /#/login
- Register — /#/register
- Forgot Password — /#/forgot-password (security-question based reset flow)

User / Account Functionality (post-authentication)

- Basket — /#/basket
- Order History — /#/order-history
- User Profile — /profile (role, username, avatar upload / image-URL linking)
- Privacy & Security menu — saved addresses, payment options, digital wallet, GDPR data export and data-erasure requests

Administrative Functionality

- Administration panel — /#/administration (registered users + customer feedback moderation)
- Accounting — /#/accounting (returned 403 Forbidden for the authenticated non-privileged test account, which shows a role-based control exists; see Section 2.5)

Interesting Resources & Hidden / Non-Obvious Paths

- robots.txt — has a Disallow rule for /ftp, which is itself a hint that a hidden directory exists
- /ftp — legacy FTP-style directory listing that exposes backup and configuration files
- /encryptionkeys — directory listing that exposes application encryption key material
- /#/score-board — the hidden "CTF" challenge tracker. You can only find it by inspecting client-side routes, not through normal navigation
- I ran a gobuster directory brute-force against the root path (wordlist: dirb/common.txt) to make sure I hadn't missed any static directories. I filtered the results against the baseline wildcard response length (9393 bytes) to remove false positives.

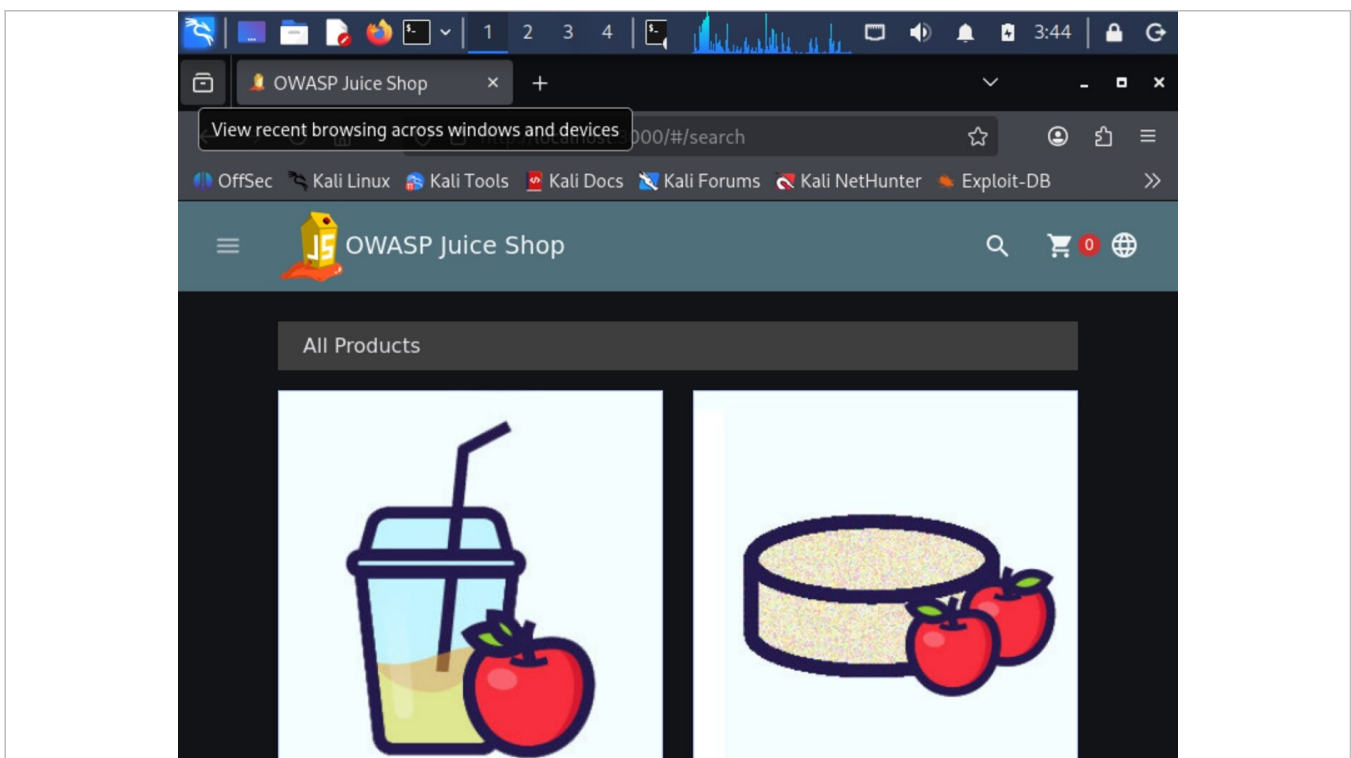


Figure 2.1 — The Juice Shop storefront at <http://localhost:3000>, confirming the application is live and interactive.

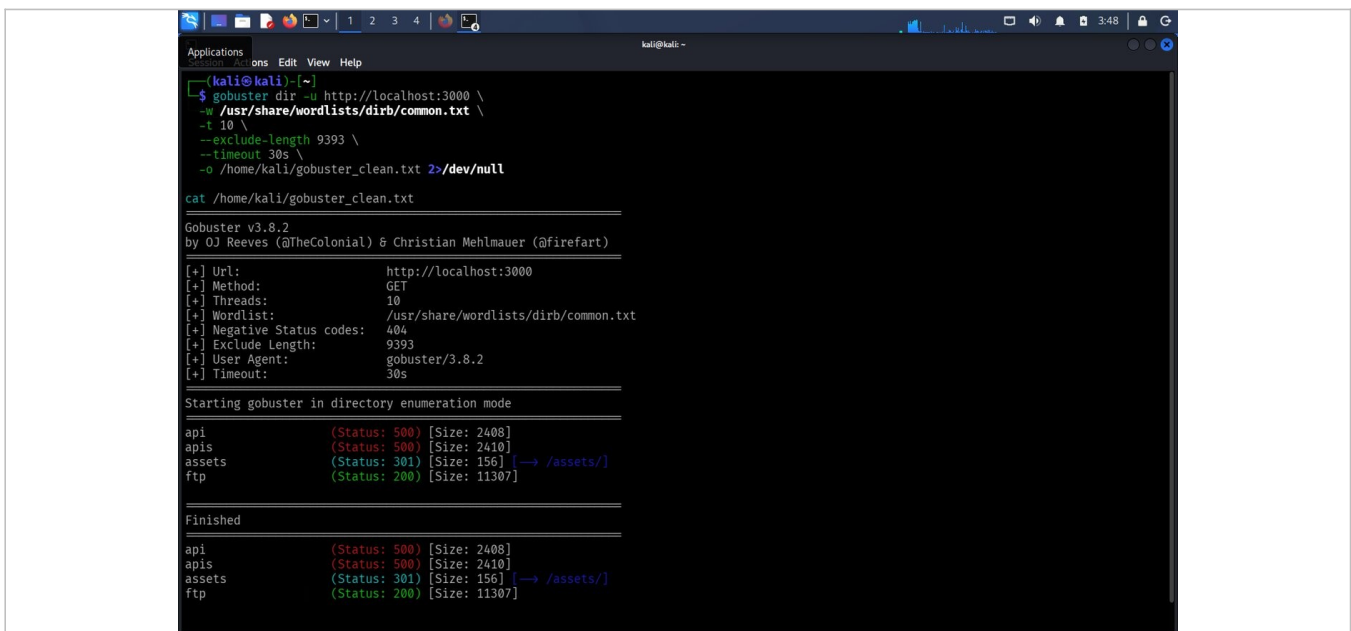


Figure 2.2 — Gobuster brute-force against the web root, used to check that no hidden static paths exist beyond the ones I found manually.

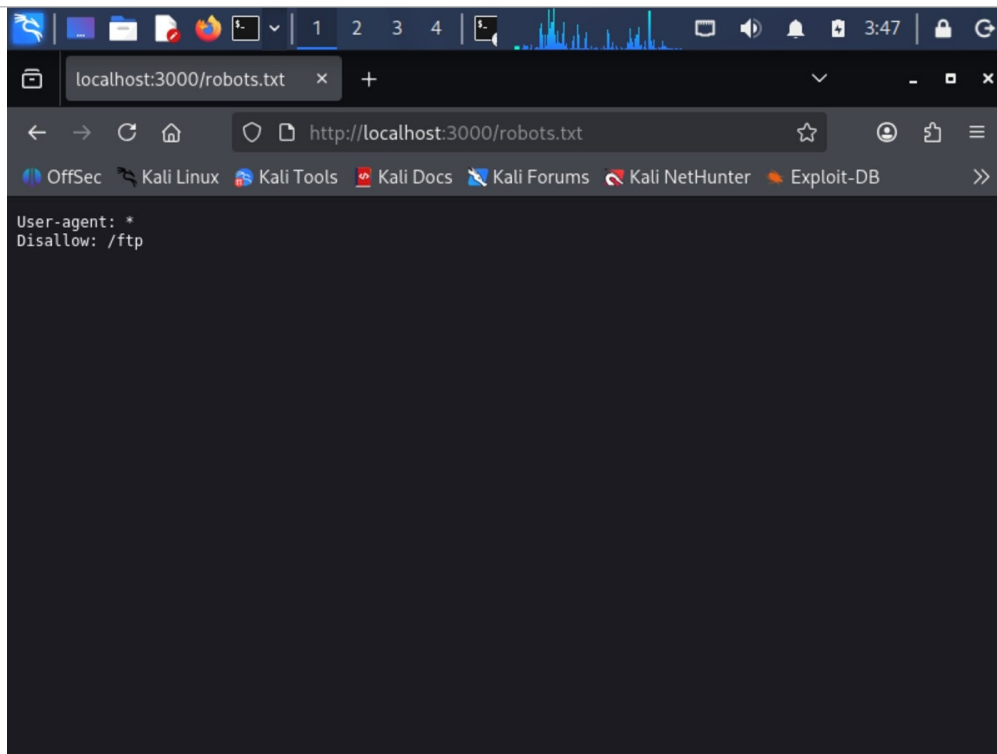


Figure 2.3 — robots.txt with a Disallow rule for /ftp, a direct pointer to a hidden, non-obvious path.

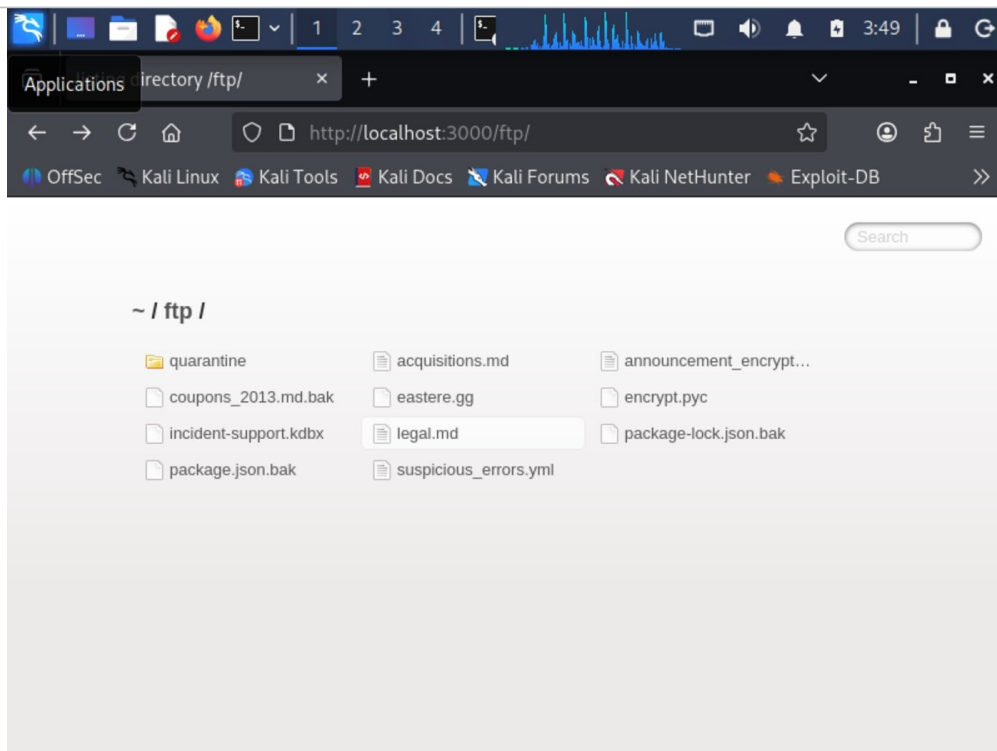
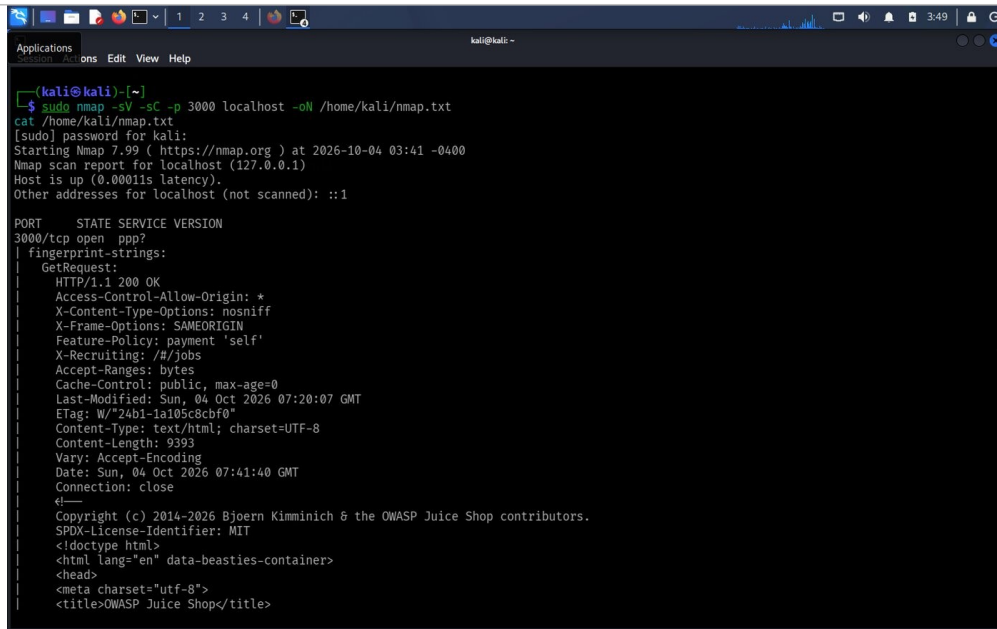


Figure 2.4 — Exposed directory listing showing backup and configuration files that should not be publicly reachable.



```
kali@kali: ~  
$ sudo nmap -sV -sC -p 3000 localhost -oN /home/kali/nmap.txt  
cat /home/kali/nmap.txt  
[sudo] password for kali:  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-04 03:41 -0400  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.00011s latency).  
Other addresses for localhost (not scanned): ::1  
  
PORT      STATE SERVICE VERSION  
3000/tcp  open  ppp?      
| fingerprint-strings:  
|_  GetRequest:  
|_    HTTP/1.1 200 OK  
|_    Access-Control-Allow-Origin: *  
|_    X-Content-Type-Options: nosniff  
|_    X-Frame-Options: SAMEORIGIN  
|_    Feature-Policy: payment 'self'  
|_    X-Recruiting: /#/jobs  
|_    Accept-Ranges: bytes  
|_    Cache-Control: public, max-age=0  
|_    Last-Modified: Sun, 04 Oct 2026 07:20:07 GMT  
|_    ETag: W/"24b1-1a105c8cbf0"  
|_    Content-Type: text/html; charset=UTF-8  
|_    Content-Length: 9393  
|_    Vary: Accept-Encoding  
|_    Date: Sun, 04 Oct 2026 07:41:40 GMT  
|_    Connection: close  
|_    <!--  
|_    Copyright (c) 2014-2026 Bjoern Kimminich & the OWASP Juice Shop contributors.  
|_    SPDX-License-Identifier: MIT  
|_    <!doctype html>  
|_    <html lang="en" data-beasties-container>  
|_    <head>  
|_    <meta charset="utf-8">  
|_    <title>OWASP Juice Shop</title>
```

Figure 2.5 — Nmap scan confirming the single exposed service: Node.js/Express on TCP 3000.

2.2 API Discovery

The Swagger / OpenAPI documentation bundled with Juice Shop (at /api-docs) turned out to be the fastest way to see the whole API. The application effectively hands an attacker a map of its own surface. I cross-checked it against the calls the app actually makes during normal browsing (captured in browser DevTools → Network) and against direct cURL requests.

Discovered Endpoint Groups

- Authentication — POST /rest/user/login, POST /rest/user/register, POST /rest/user/reset-password
- User / Account — GET /api/Users, GET/POST /api/SecurityQuestions, GET/POST /api/SecurityAnswers
- Products — GET /api/Products (public and unauthenticated; returns the full catalogue with prices and internal IDs)
- Basket / Orders — GET/PUT /api/BasketItems, GET /rest/basket/:id, order history retrieval
- Payment / Address — /api/Cards, /api/Addresses (card and address book management)
- Challenges / Scoreboard — GET /api/Challenges?name=... (returns challenge metadata, including completion status, with no authentication)
- GDPR / Data Privacy — data export (JSON/PDF/Excel) and data-erasure request endpoints under Privacy & Security

HTTP Methods & Authentication Requirements

I ran an OPTIONS pre-flight sweep across representative endpoints (/api/Products, /api/Users, /api/BasketItems, /api/SecurityAnswers, /rest/user/login and /rest/basket/1). Every one returned the same CORS policy: Access-Control-Allow-Methods: GET, HEAD, PUT, PATCH, POST, DELETE and Access-Control-Allow-Origin: *. In practice, the server tells any origin that full CRUD (including DELETE) is available on resource endpoints, with no method-level restriction at the CORS layer.

Authentication, however, is not enforced consistently. GET /api/Products and GET /api/Challenges respond fine with no Authorization header at all. GET /api/Users and GET /rest/basket/:id correctly reject unauthenticated requests with 401 Unauthorized. A DELETE request to /api/Products/1 was rejected both with no token (401 Unauthorized) and with a user-level Bearer token, so product deletion does enforce some authorization. But the mix of public, session-gated and apparently role-gated endpoints is a key finding in itself: the authorization model isn't applied uniformly.

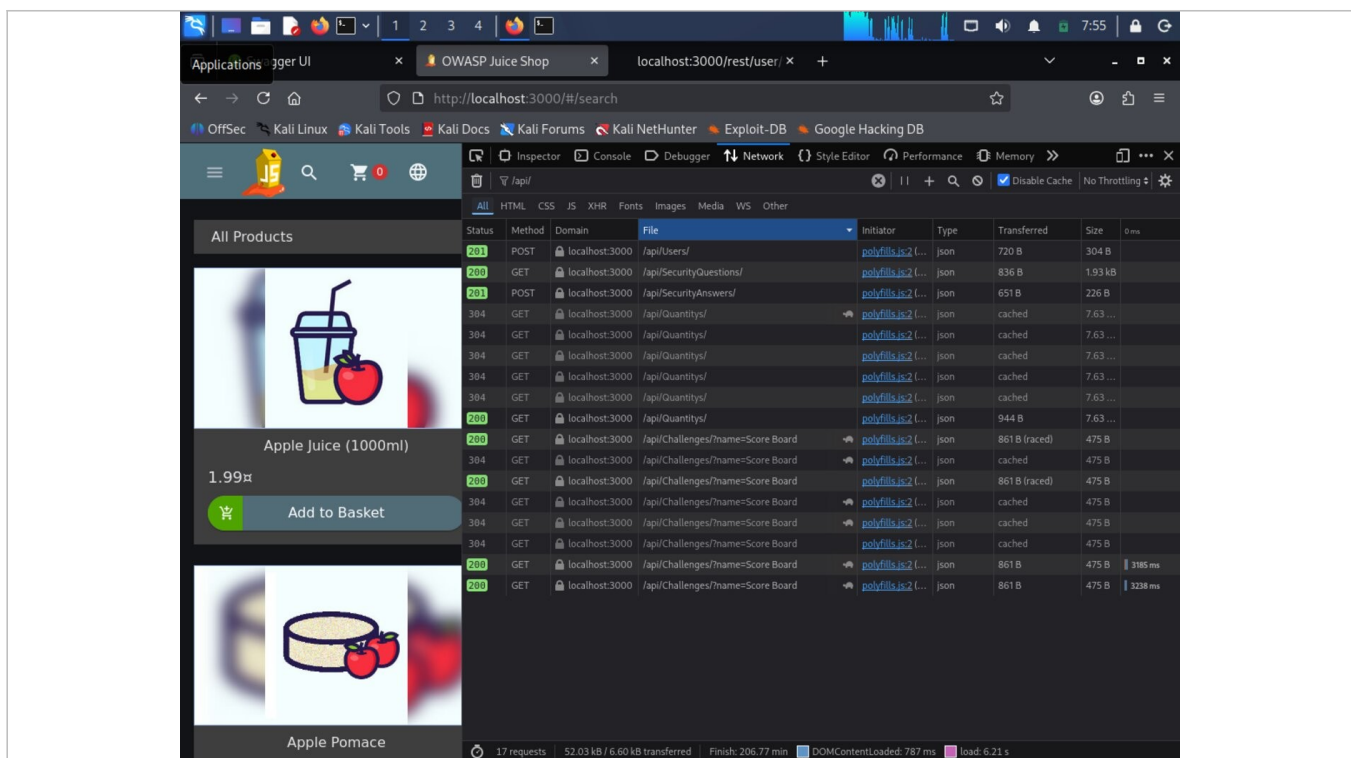


Figure 2.6 — Swagger/OpenAPI documentation disclosing the application's full API surface.

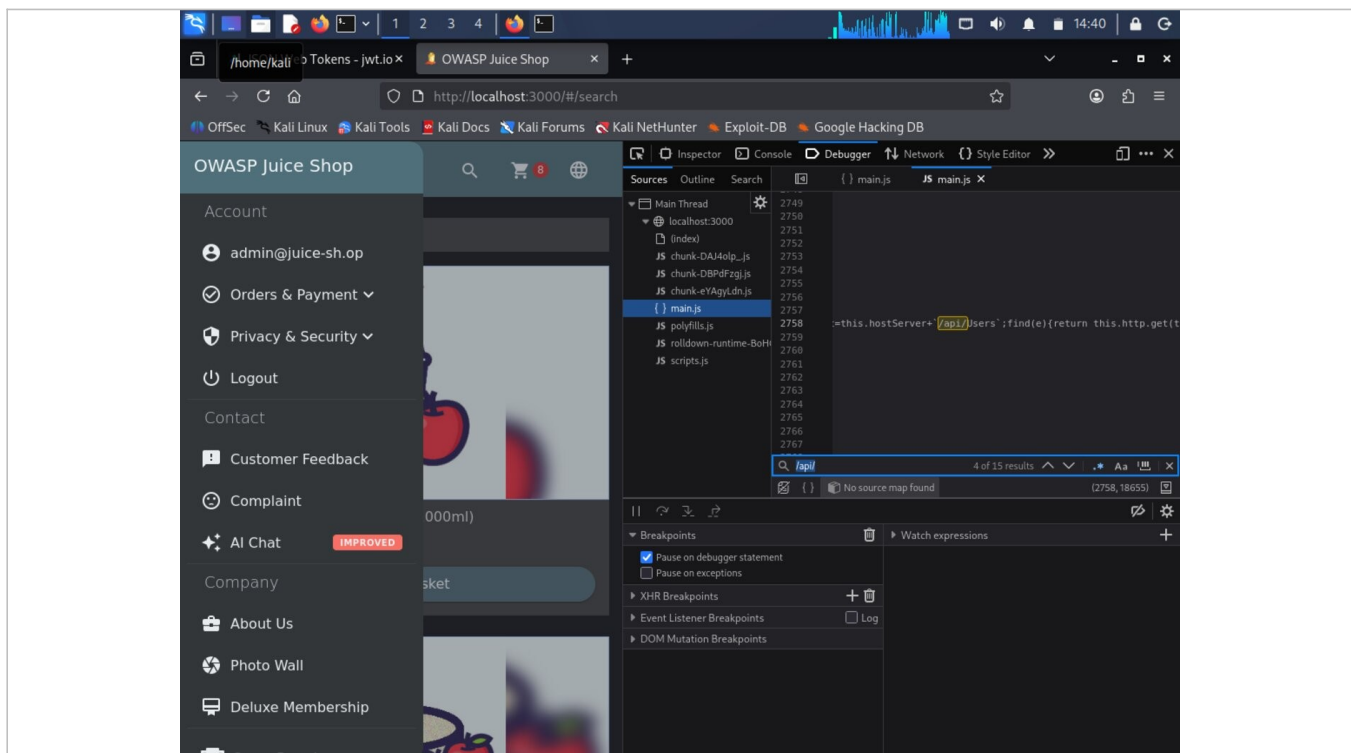


Figure 2.7 — GET /api/Users returning 401 Unauthorized with no Authorization header, confirming the endpoint exists and needs a session.

[illegible]

2.3 Technology Fingerprinting

Confidential — Training Use Only | Page 12 of 28

Frontend Framework	Angular (SPA, hash-based routing /#/...)	ng- component selectors & attributes visible in rendered DOM
Backend Runtime	Node.js	X-Powered-By: Express response header
Backend Framework	Express.js	X-Powered-By: Express header on every response
ORM / Database	Sequelize ORM over SQLite	package.json dependencies; .sqlite artifacts referenced in app internals
API Documentation	Swagger / swagger-ui-express	package.json dependency + live /api-docs UI
Authentication	JWT (JSON Web Token), Bearer scheme	Authorization: Bearer <token> header; token decodable in a JWT debugger
Containerization	Docker (official image)	docker image manifest / layer inspection
Other Notable Deps	body-parser, cors, helmet-style security headers, multer (uploads)	package.json dependency grep

```

kali@kali: ~
$ whatweb -v http://localhost:3000 | tee /home/kali/whatweb.txt
WhatWeb report for http://localhost:3000
Status      : 200 OK
Title       : OWASP Juice Shop
IP          : <Unknown>
Country     : <Unknown>

Summary    : HTML5, Script[module], UncommonHeaders[access-control-allow-origin,x-content-type-options,feature-policy,x-recruiting], X-Frame-Options[SAMEORIGIN]

Detected Plugins:
[ HTML5 ]
  HTML version 5, detected by the doctype declaration

[ Script ]
  This plugin detects instances of script HTML elements and returns the script language/type.
  String      : module

[ UncommonHeaders ]
  Uncommon HTTP server headers. The blacklist includes all the standard headers and many non standard but common ones. Interesting but fairly common headers should have their own plugins, eg. x-powered-by, server and x-aspnet-version. Info about headers can be found at www.http-stats.com
  String      : access-control-allow-origin,x-content-type-options,feature-policy,x-recruiting (from headers)

[ X-Frame-Options ]
  This plugin retrieves the X-Frame-Options value from the HTTP header. - More Info:
  http://msdn.microsoft.com/en-us/library/cc288472%28VS.85%29.aspx

```

Figure 2.10 — WhatWeb scan summarizing the detected technologies in a single pass.

```

kali@kali: ~
$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
d8adb5a99975   bkimminich/juice-shop   "/nodejs/bin/node /j..."   About a minute ago   Up 59 seconds   0.0.0.0:3000->3000/tcp

kali@kali: ~
$ curl -I http://localhost:3000
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Accept-Ranges: bytes
Cache-Control: public, max-age=0
Last-Modified: Sat, 26 Sep 2026 21:59:24 GMT
ETag: W/"24b1-1a0dfbaea86"
Content-Type: text/html; charset=UTF-8
Content-Length: 9393
Vary: Accept-Encoding
Date: Sat, 26 Sep 2026 22:00:39 GMT
Connection: keep-alive
Keep-Alive: timeout=5

```

Figure 2.11 — The X-Powered-By: Express response header: a direct, high-confidence backend fingerprint that should ideally be suppressed in production.

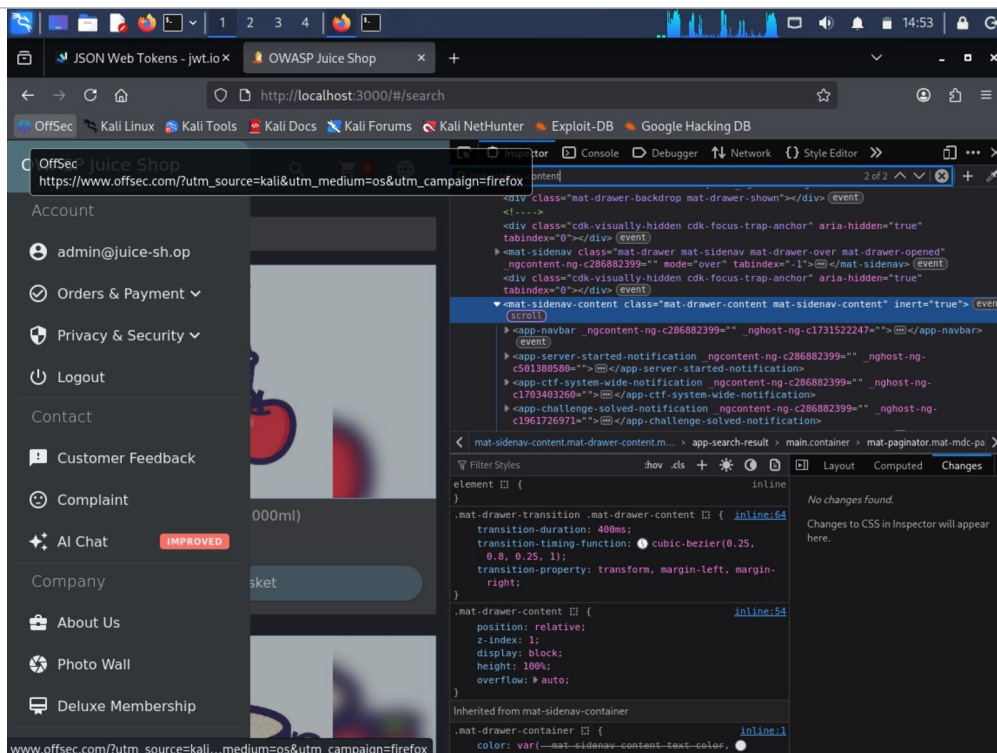


Figure 2.12 — Angular component selectors in the rendered DOM, confirming the Angular SPA front end.

I also looked directly at the application's Docker image and the legacy FTP directory. The image layers and an FTP-exposed package-lock.json each independently back up the sqlite3 / sequelize / express / swagger-ui-express stack. I also recovered an exposed ctf.key file and an "AI config" directory listing. An unauthenticated client has no reason to reach either of them, so I list both as findings in Section 2.6.

2.4 Client-Side Analysis

I used browser DevTools (the Sources, Network and Application tabs) to inspect the Angular client bundle, the storage mechanisms and the request headers, looking for anything that widens the attack surface beyond what normal navigation shows.

- Client-side route table: the compiled JS bundle holds the full Angular route configuration. Routes such as /administration and /accounting exist in the client even where server-side RBAC correctly blocks access.
- Authentication token storage: the JWT issued at login sits in the browser's localStorage rather than in an HttpOnly cookie. Any client-side script can read it, which makes an XSS-based token theft far more damaging.
- Authorization header format: requests to protected endpoints carry Authorization: Bearer <JWT>. I decoded the token in a JWT debugger and could read the user's id, email and role claims in plaintext (JWTs are signed, not encrypted).
- sessionStorage / cookies: I also saw a lightweight session cookie and a few small sessionStorage entries, used alongside the bearer token for some requests. This dual authentication-state pattern is worth testing for session fixation and token leakage.
- search.ts / route source inspection: TypeScript source artifacts reachable through source maps show how query parameters are handled internally (for example, a Sequelize raw-query pattern in the search route). That is a strong lead for injection testing in a later, authorized phase.

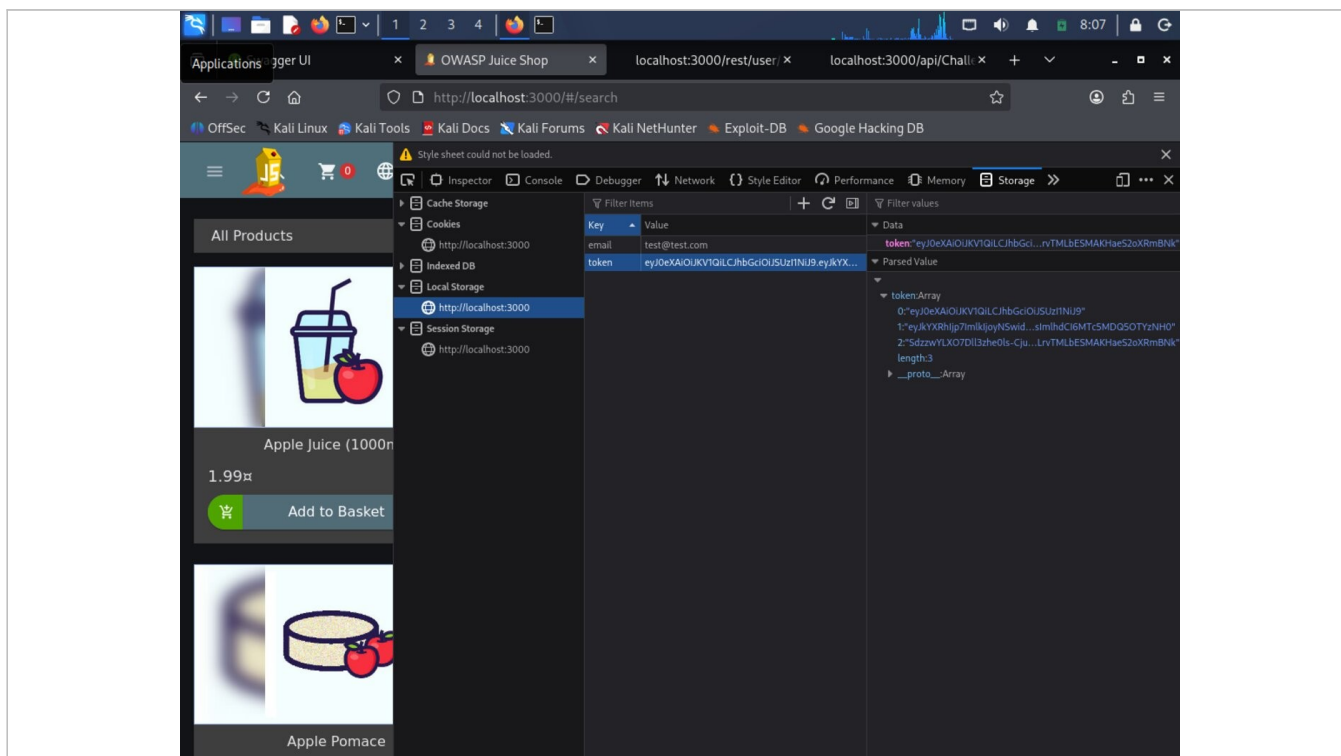


Figure 2.14 — Browser localStorage showing the JWT authentication token stored client-side.

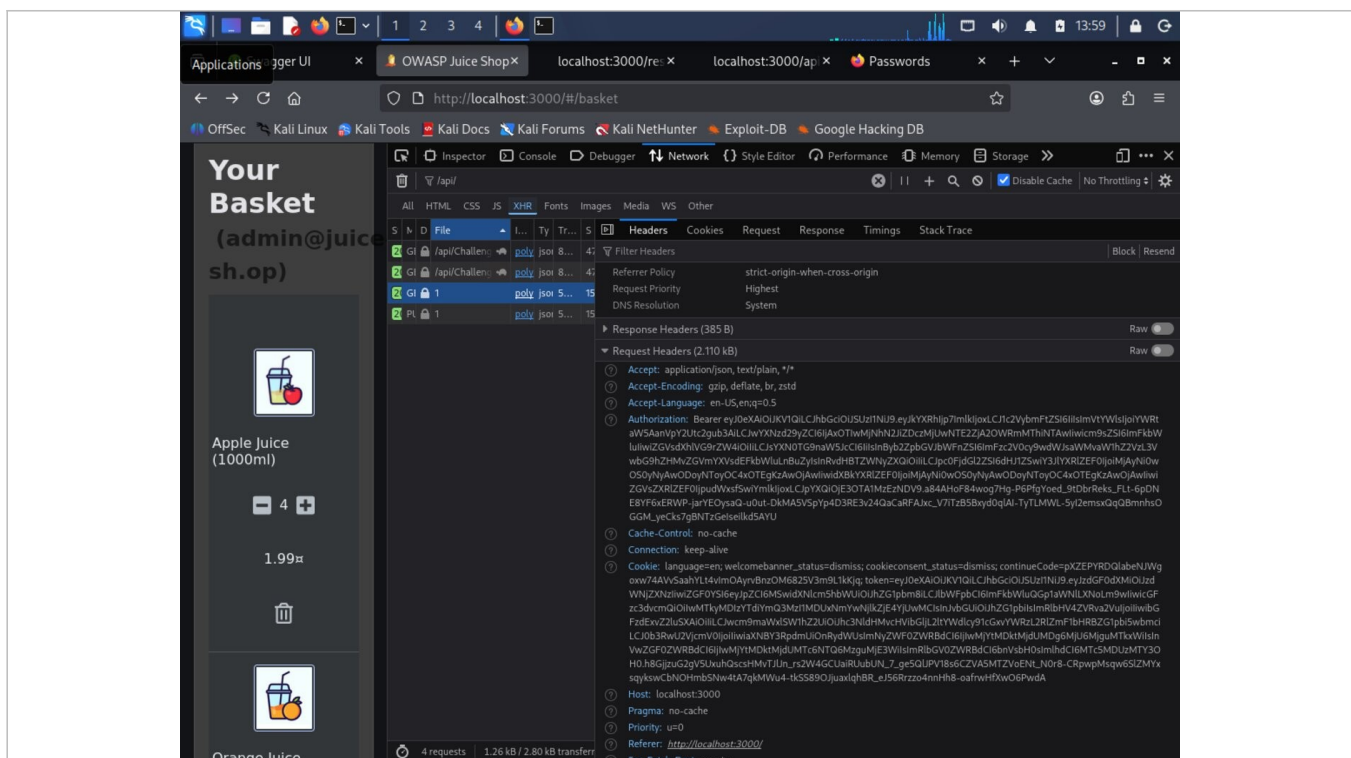


Figure 2.15 — Authorization: Bearer header captured on an authenticated API request.

2.5 Access Control, Admin Panel & Authorization Testing

Role-based access control (RBAC) is one of the most consequential parts of an attack surface, so during discovery I deliberately probed the boundary between anonymous, authenticated-user and administrative access.

- I could reach the administration panel (/#/administration) after logging in with a known administrative test account. It shows the full list of registered users (including email addresses) and raw, unmoderated customer feedback. Some of that feedback already contains user-submitted, social-engineering-style content, which is worth flagging on its own.
- The accounting page (/#/accounting) correctly returned 403 Forbidden for a standard authenticated (non-admin) session. That confirms at least one admin-only route is properly protected, which contrasts with how easily the administration panel could be reached.
- As a legitimate account owner, I used the GDPR "Data Export" feature to request a JSON export of my account data. It returned order history, review history and the account email in readable plaintext JSON. The endpoint works as documented, but it also exposes a lot of structured data if its authorization is ever weakened.

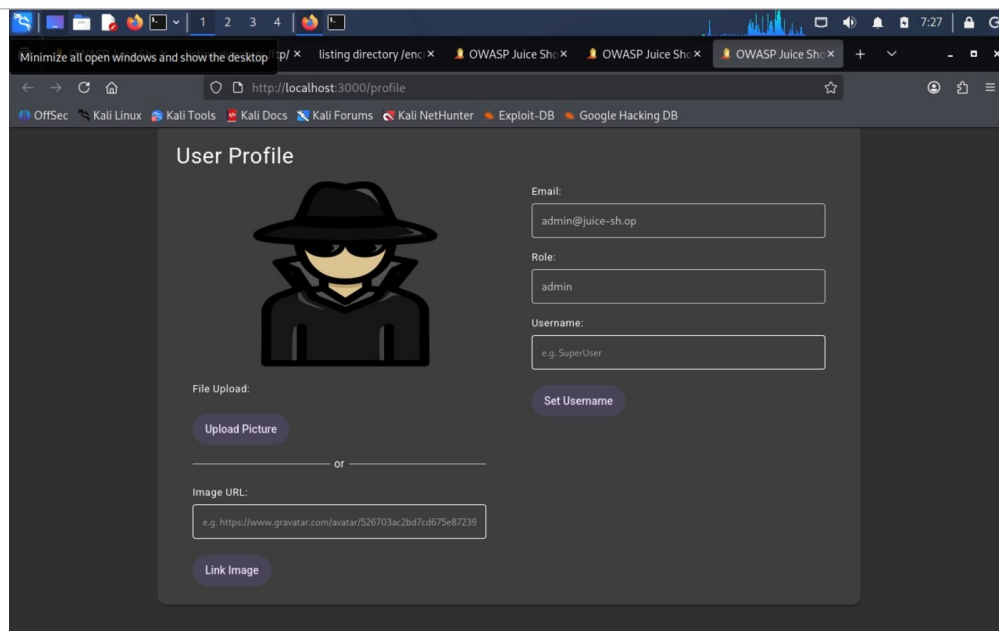


Figure 2.16 — Authenticated user profile confirming an "admin" role claim was obtainable and used for administrative testing.

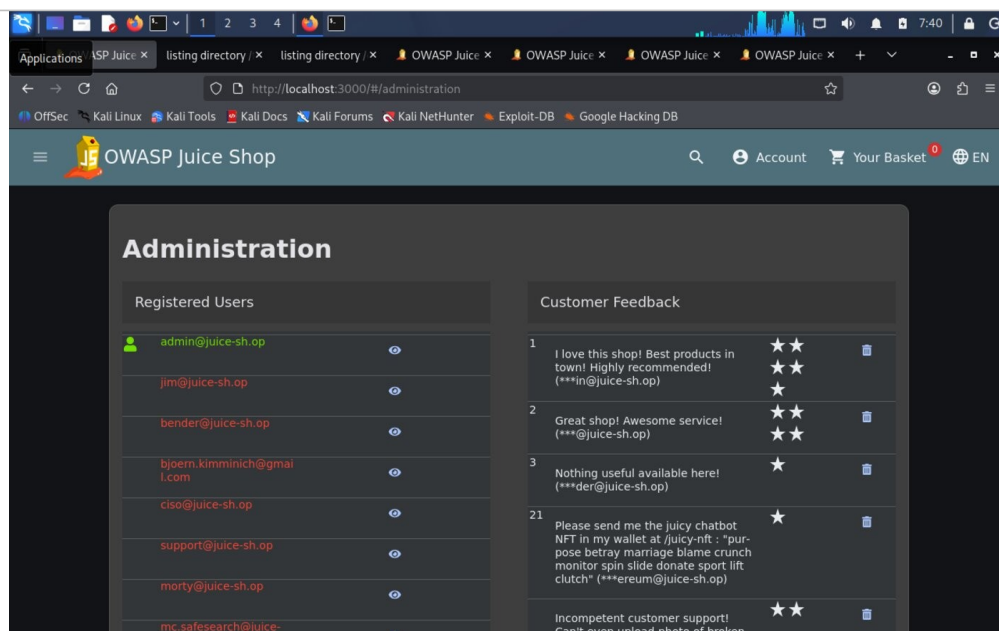


Figure 2.17 — Administration panel showing the full list of registered users and unmoderated customer feedback.

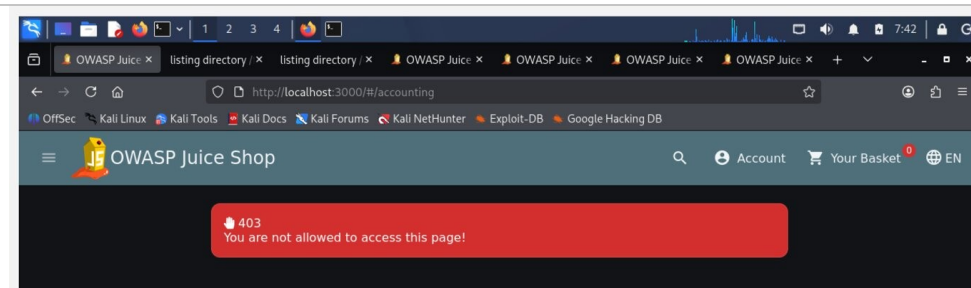


Figure 2.18 — `/#/accounting` correctly returning 403 Forbidden for a non-privileged session, showing that RBAC is enforced on some administrative routes but not all.

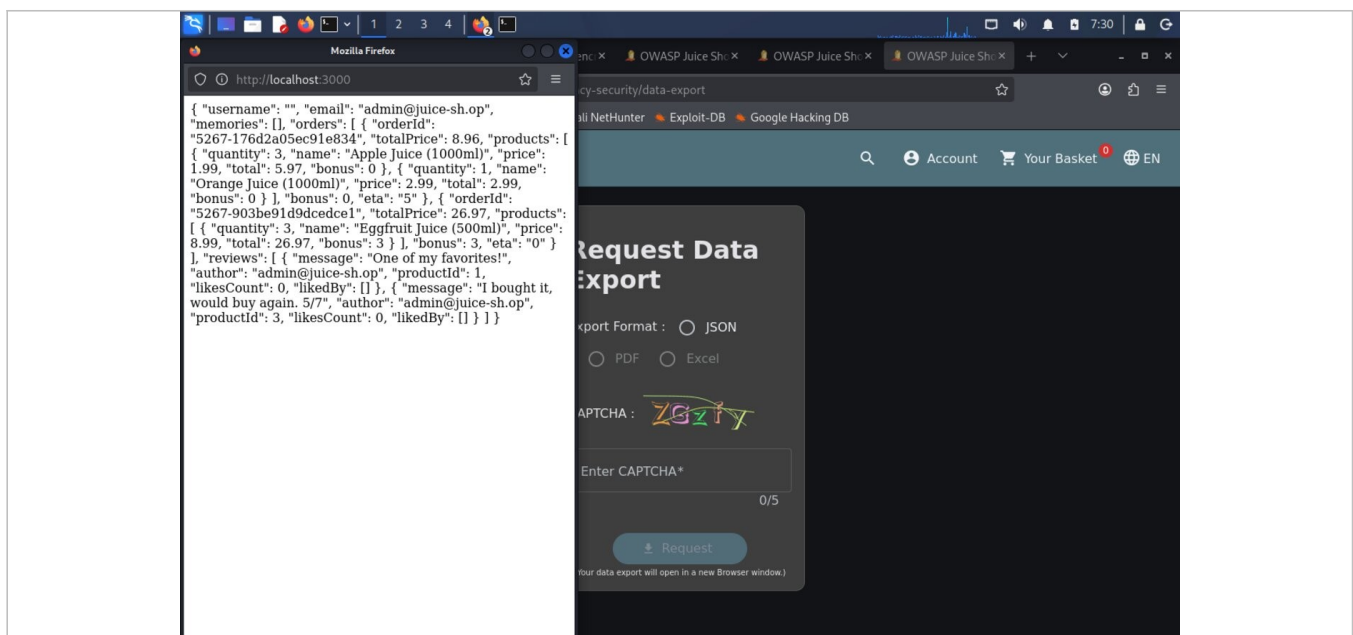
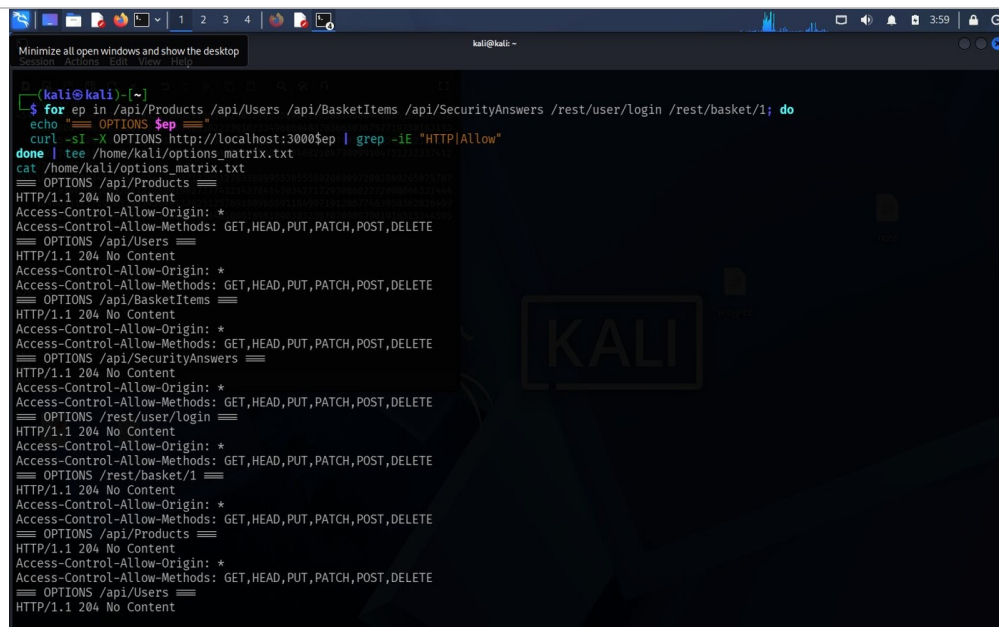


Figure 2.19 — GDPR data-export response returning structured account, order and review data as JSON.

2.6 HTTP Method Enumeration & Exposed Secrets

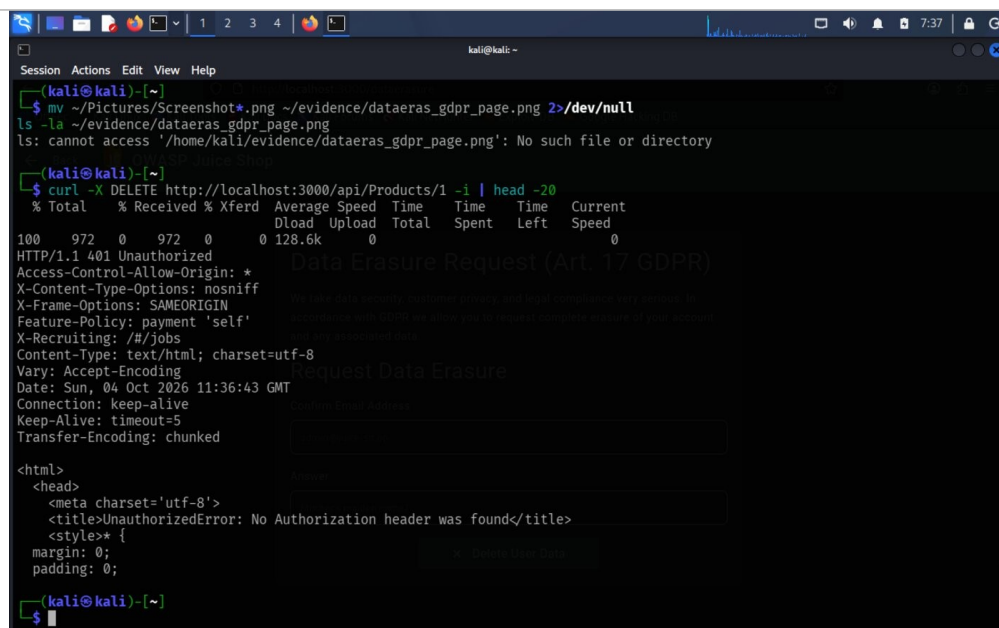
A scripted OPTIONS sweep (see Figure 2.20) confirmed that the API-wide CORS policy advertises GET, HEAD, PUT, PATCH, POST and DELETE on every resource endpoint I tested, with a wildcard Access-Control-Allow-Origin. I then sent a direct DELETE /api/Products/1 request twice: once with no authentication (401 Unauthorized, so the method is wired up but gated) and once with a valid but under-privileged Bearer token (also rejected). That is useful negative evidence: write and delete operations are not trivially exposed, even though CORS advertises them broadly.

Separately, the legacy /ftp directory and an /encryptionkeys-style listing (Section 2.1) led me to an exposed ctf.key file and an "AI configuration" directory listing. Neither should be reachable by an anonymous client in a hardened deployment. This is an intentionally vulnerable training app, but the pattern (stale backup or key material sitting under a predictable path) is one of the most common real-world ways in, so I flag it here.



```
(kali@kali)~$ for ep in /api/Products /api/Users /api/BasketItems /api/SecurityAnswers /rest/user/login /rest/basket/1; do
echo "=== OPTIONS $ep ==="
curl -sI -X OPTIONS http://localhost:3000$ep | grep -iE "HTTP|Allow"
done | tee /home/kali/options_matrix.txt
cat /home/kali/options_matrix.txt
=== OPTIONS /api/Products ===
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
=== OPTIONS /api/Users ===
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
=== OPTIONS /api/BasketItems ===
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
=== OPTIONS /api/SecurityAnswers ===
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
=== OPTIONS /rest/user/login ===
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
=== OPTIONS /rest/basket/1 ===
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
=== OPTIONS /api/Products ===
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
=== OPTIONS /api/Users ===
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
```

Figure 2.20 — OPTIONS/CORS sweep confirming GET, HEAD, PUT, PATCH, POST and DELETE are advertised as allowed methods across core API resources.



```
(kali@kali)~$ mv ~/Pictures/Screenshot*.png ~/evidence/dataaras_gdpr_page.png 2>/dev/null
ls -la ~/evidence/dataaras_gdpr_page.png
ls: cannot access '/home/kali/evidence/dataaras_gdpr_page.png': No such file or directory

(kali@kali)~$ curl -X DELETE http://localhost:3000/api/Products/1 -i | head -20
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           % Done    0     0     0    0      0         0      0
100    972    0   972    0    128.6k    0      0      0
HTTP/1.1 401 Unauthorized
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Content-Type: text/html; charset=utf-8
Vary: Accept-Encoding
Date: Sun, 04 Oct 2026 11:36:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5
Transfer-Encoding: chunked

<html>
<head>
  <meta charset='utf-8'>
  <title>UnauthorizedError: No Authorization header was found</title>
  <style>* {
margin: 0;
padding: 0;

```


Part 3 — Attack-Surface Diagram

The diagram below pulls every component from Part 2 into one architecture-level view. It follows an unauthenticated outside attacker from network-level reconnaissance, through the Angular presentation layer and its client-side artifacts, into the Express/Node.js API layer, and finally to the SQLite data store. The two red paths mark the access-control weaknesses from Sections 2.5 to 2.7 (the reachable administration surface and the basket IDOR), because they are the shortest realistic route from "anonymous visitor" to "sensitive data or admin function."

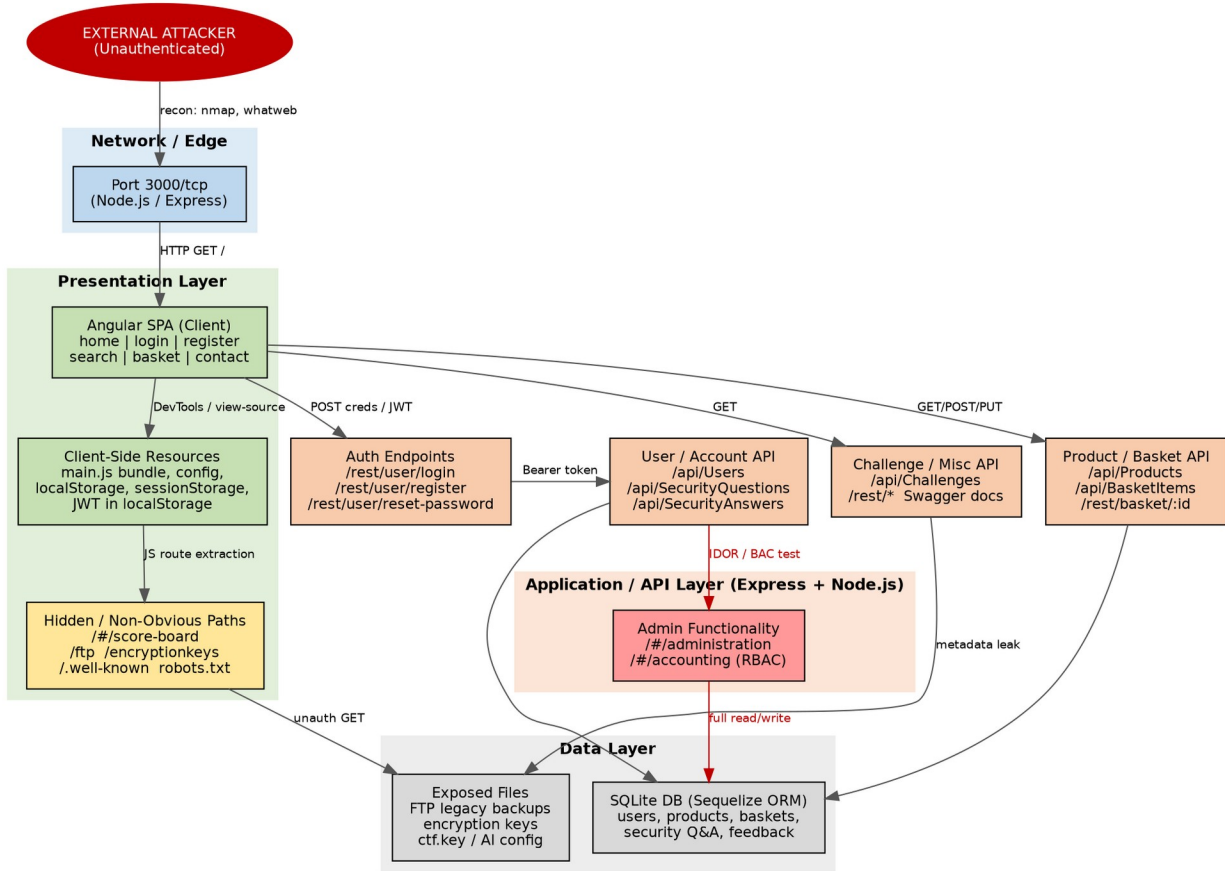


Figure 3.1 — OWASP Juice Shop attack-surface diagram: component relationships and the main attacker path (red = access-control weak points).

Part 4 — Attack-Surface Inventory

The table below brings together every asset and endpoint I found during this assessment, with its access method, underlying technology, security relevance and a priority on a five-level scale (Critical / High / Medium / Medium-Low / Low). The colour coding is there to make triage quick in a follow-on engagement.

Asset / Endpoint	Type	Access	Method(s)	Technology	Security Relevance	Priority
/#/administration	Web	Auth (role-gated, weakly enforced)	GET	Angular + Express	Exposes the full user list and raw feedback to any session with the admin role; no secondary verification	CRITICAL
/rest/basket/:id (IDOR)	API	Auth (session, not ownership-checked)	GET/PUT	Express + Sequelize	Returns and modifies another user's basket contents (Broken Access Control, OWASP A01)	CRITICAL
/ftp , /encryptionkeys	Web	Public (unauthenticated)	GET	Static file serving	Legacy directory exposes backup files and encryption key material	HIGH
/api/Users	API	Auth required (401 if missing)	GET/POST	Express + Sequelize	Holds user account data; correctly gated, but a high-value target for credential and role data	HIGH
/api/Products/:id (DELETE)	API	Auth + role required	DELETE	Express + Sequelize	Write/delete capability is advertised through CORS on every resource. Correctly rejected in testing, but a key area for privilege-escalation testing	HIGH
JWT in localStorage	Client	N/A (client-side storage)	N/A	Angular / browser storage	Any script in the page can read the token, so an XSS flaw would mean full session takeover	HIGH
/api/Challenges	API	Public (no auth)	GET	Express	Discloses challenge/CTF completion data and internal naming without authentication	MEDIUM
/#/accounting	Web	Auth, admin-role (enforced)	GET	Angular + Express	Correctly returns 403 for a non-	MEDIUM

					admin session, which confirms RBAC is only partly enforced	
/api-docs (Swagger)	Web/API	Public	GET	swagger-ui-express	Documents the entire API surface: handy for defenders, a map for attackers	MEDIUM
GDPR Data Export	API	Auth (account owner)	GET/POST	Express + Sequelize	Returns full structured account, order and review data as JSON, PDF or Excel	MEDIUM
/#/forgot-password	Web	Public	GET/POST	Angular + Express	Security-question-based reset flow; risk of question enumeration	MEDIUM-LOW
/#/score-board	Web	Public (hidden, non-indexed)	GET	Angular	Non-obvious path to the CTF/challenge tracker, which isn't linked from normal navigation	MEDIUM-LOW
X-Powered-By: Express header	Web	Public	N/A	Express default header	Minor information disclosure that helps with technology fingerprinting	LOW
/robots.txt	Web	Public	GET	Static	Reveals the /ftp path through its Disallow rule	LOW
/api/Products	API	Public	GET	Express + Sequelize	Public catalogue data; low sensitivity by design	LOW

Part 5 — Prioritized Testing Plan

I triaged every component I found on a five-level priority scale (Critical, High, Medium, Medium-Low and Low). The rating reflects the potential security impact, how easily an unauthenticated or low-privileged attacker can reach the component, and how sensitive the data or function behind it is. Below is the order I would recommend for a follow-on, authorized penetration test.

CRITICAL PRIORITY

- Basket IDOR (/rest/basket/:id and the BasketItems API): verify how far cross-user data access goes, and whether write operations (changing another user's basket or order) are also possible.
- Administration panel (/#/administration): confirm exactly which authorization check protects this route, and whether a standard customer account could reach it or escalate into it.

Why first: *These two findings give the most direct path from an anonymous or low-privileged session to another user's private data or to administrative functions. They are the highest-impact, lowest-effort outcomes for an attacker that I found during discovery.*

HIGH PRIORITY

- Exposed /ftp and /encryptionkeys directories: work out how sensitive each recoverable file is (backups, keys, ctf.key, AI config) and whether any key material is reused in production-style deployments.
- JWT handling: assess token storage (localStorage), the signature algorithm, expiry, and whether role claims can be tampered with or replayed.
- DELETE and other write-method authorization on /api/Products and similar endpoints: confirm the role checks hold up against privilege-escalation and parameter-tampering attempts.

Why first: *These involve credential and key material and the integrity of the authentication mechanism itself. A compromise here would undermine every other control in the application.*

MEDIUM PRIORITY

- /#/accounting and other role-gated routes: map the full set of admin-only routes and check that enforcement is consistent (one gap is already known, a sibling route that was not gated).
- GDPR data export and data erasure: test for authorization bypass (exporting another user's data) and for abuse of erasure requests.
- /api/Challenges and Swagger /api-docs: look for further information disclosure beyond what I captured in this pass.

MEDIUM-LOW PRIORITY

- /#/forgot-password security-question flow: test for question enumeration and how well it resists brute force.
- /#/score-board and other hidden or non-obvious routes: confirm nothing else is reachable beyond the challenge tracker.

LOW PRIORITY

- Informational headers (X-Powered-By, server banners): worth suppressing, but low exploitability on their own.
- robots.txt disclosure and public product catalogue endpoints: expected, low-sensitivity data that is public by design.

Recommendations

The guidance below maps directly to the findings above and follows the same order as the priorities in Part 5. It reflects standard secure-development practice and is written for the training context of this exercise.

Immediate (Critical / High)

- Enforce object-level ownership checks on every basket, order and user-data endpoint. Never trust a client-supplied ID on its own; always verify that it belongs to the authenticated user.
- Apply one centrally enforced RBAC middleware across all administrative routes instead of checking route by route. That would remove the kind of inconsistency I saw between /administration and /accounting.
- Remove the legacy /ftp and /encryptionkeys directories or put proper access control on them, and rotate any key material that was exposed.
- Move authentication tokens out of localStorage and into HttpOnly, Secure, SameSite cookies, so any future XSS finding has less impact.

Short-Term (Medium / Medium-Low)

- Re-validate authorization on the GDPR export and erasure endpoints against the authenticated user, regardless of any identifiers the client supplies.
- Restrict Swagger/OpenAPI documentation, or require authentication for it, in production-equivalent environments.
- Add rate limiting and generic error responses to the password-reset and security-question flow, to resist enumeration and brute-force attacks.

Longer-Term / Hardening (Low & General)

- Remove or customize framework fingerprinting headers (X-Powered-By, default error pages) to slow down automated reconnaissance.
- Adopt a Content-Security-Policy and Subresource Integrity to limit the damage from any client-side injection.
- Make an attack-surface review like the one in this report a regular part of each release cycle, so new routes and endpoints are catalogued and risk-assessed before they ship.

Conclusion

This exercise produced a structured, evidence-backed attack-surface map of OWASP Juice Shop, using the same reconnaissance approach an attacker would take: verifying the environment, manual and automated web discovery, API enumeration through self-documentation and direct probing, technology fingerprinting, and client-side artifact analysis. The resulting inventory, diagram and prioritization give a clear, risk-ranked starting point for a later authorized penetration test. The basket IDOR and the administration-panel exposure are the most urgent items to validate and fix first.

All activity took place in a self-contained, locally hosted training environment, with no exploitation beyond non-destructive verification. That is consistent with the restrictions in the Week 1 task brief and with the liability disclaimer at the front of this report.



Thank you for reading.

Week 1 — Attack Surface Mapping of OWASP Juice Shop

This report is my record of the first week of the Cronova Solutions Cybersecurity Internship. I set out to map OWASP Juice Shop the way an attacker would — finding every page, API and hidden path — without exploiting anything. I learned a lot along the way, and I hope it reads as clearly as it felt to piece together.

A QUICK NOTE ON SCOPE

Everything here was done in my own isolated lab. No real systems were touched, and nothing was exploited — I only looked, verified and wrote it down.

PLEASE KEEP IT INTERNAL

This is a confidential, training-use document. Please don't copy or share it with anyone outside Cronova Solutions.

Muhammad Ali Mobeen

Cybersecurity Intern, Cronova Solutions · Ref. CSL-INT-010 · October 2026

Questions or feedback? I'd love to hear from you.

cronovasolutions.tech · contact@cronovasolutions.tech

